UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FCFM
FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

UNIVERSIDAD AUTONOMA DE NUEVO LEON

FACULTAD DE CIENCIAS FISICO MATEMATICAS

ALGORITMIA Y OPTIMIZACION

TAREA 2

CESAR YAHIR ALONSO REYES

2069375

25/10/26

ANALISIS DEL EJERCICIO 1

INSTRUCCIONES:

- Indicación: Ejecuta estimate_pi con 1, 2, 4 y 8 procesos.
- Visualización: Grafica la aceleración (speedup) vs. número de procesos
- Análisis: ¿Por qué la aceleración no es perfectamente lineal? (Considera sobrecostos de comunicación, sincronización o la ley de Amdahl).

Luego de ejecutar el código un par de veces y revisar los resultados de la aceleración paralela para la estimación de π, lo primero que se nota a simple vista es que la línea del *speedup* no crece de forma lineal como se esperaría en teoría, sino que empieza a disminuir el crecimiento, especialmente después de 4 procesos.

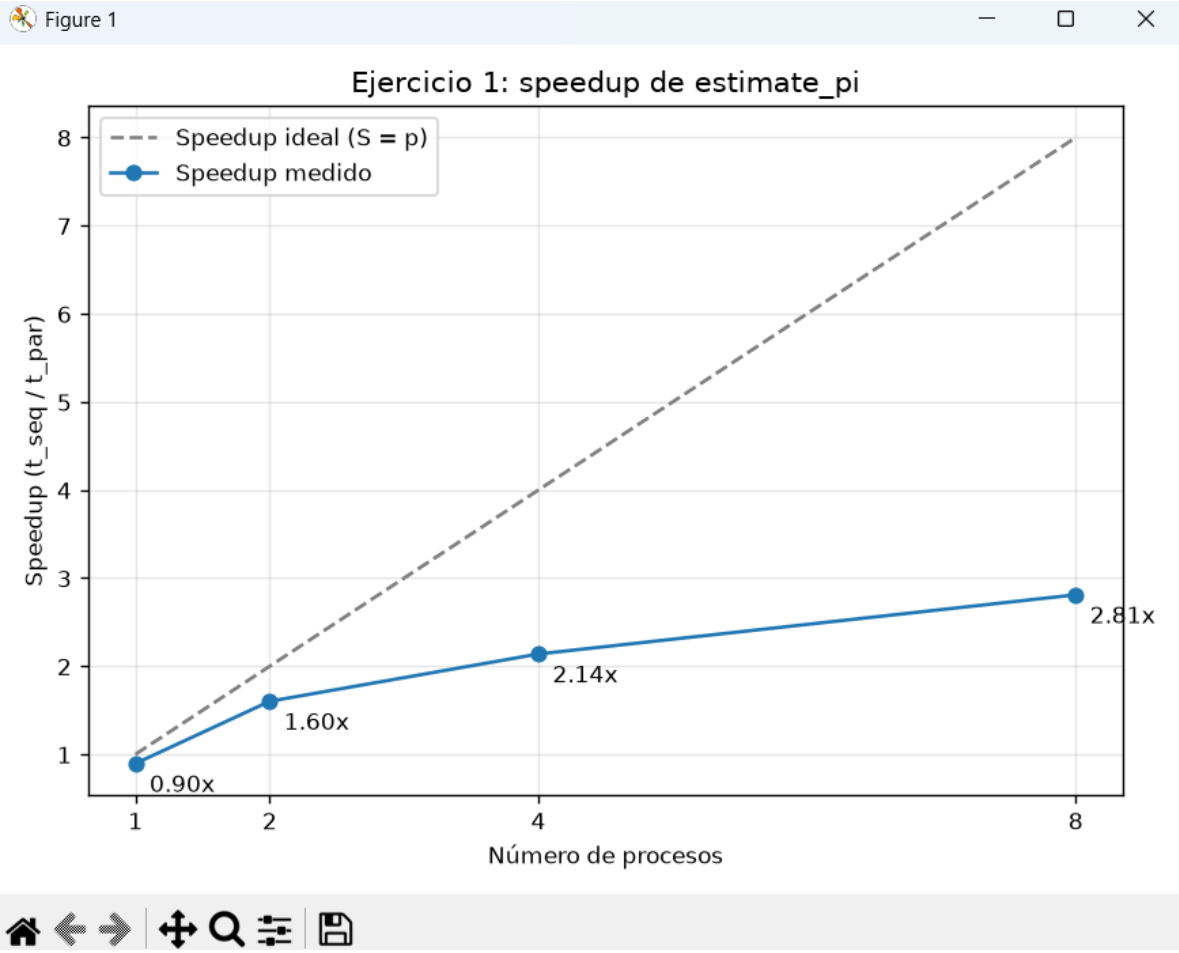
Luego del análisis a fondo de los datos, investigando por qué pasa esto encontré que esto pasa por una combinación de factores propios de cómo funciona la computación paralela en la práctica:

Empezando con que chocar contra la ley de Amdahl es prácticamente inevitable, esta ley describe el límite teórico de aceleración de un sistema cuando solo una parte del mismo puede mejorarse o paralelizarse, en este caso aunque calcular los puntos de Monte Carlo es un trabajo totalmente independiente, el programa entero tiene una parte que no se puede repartir como crear el Pool, arrancar los procesos, enviar las instrucciones y recolectar los resultados finales.

Un detalle que me llama la atención es lo que pasa cuando usamos un solo proceso, al correr la prueba con $p = 1$, el programa fue un poco más lento que hacerlo de forma secuencial, luego de revisar y consultar he llegado a la conclusión de que esto ocurre por el sobrecosto de creación conocido como “overhead”, ya que Inicializar el Pool toma una fracción de tiempo que no aporta nada útil si solo usas un proceso.

Durante la investigación me di cuenta de que también influye bastante la sincronización y el balance de carga. Al usar pool.map con 8 tareas grandes, el programa principal se queda esperando obligatoriamente a que termine el proceso más lento, en teoría, con 8 procesos el tiempo debería ser mínimo, pero en la práctica medimos más tiempo del esperado debido a los pequeños retrasos y los tiempos muertos de espera entre los hilos.

GRAFICA SPEEDUP VS CANTIDAD DE PROCESOS



Aquí podemos ver grafica de una ejecución donde la línea azul representa el crecimiento del speedup y es posible observar como el crecimiento se reduce cada vez más, la línea punteada representa el speedup ideal teórico el cual crece linealmente por cantidad de procesos

ANALISIS DEL EJERCICIO 2

CODIGO DE LA MOCHILA ORDENADO POR PESO:

```
def obtener_peso(objeto):
    return objeto[2]

def greedy(objetos, cap):
    total_v, total_w, llevar = 0, 0, []
    for name, v, w in sorted(objetos, key=obtener_peso):
        if total_w + w <= cap:
            llevar.append(name); total_v += v; total_w += w
    return total_v, total_w, llevar
```

CASO EN EL QUE ES PEOR EN COMPARACION A ELEGIR POR RATIO:

objeto	Valor	Peso	Valor/Peso
A	45	40	1.125
B	65	50	1.3
C	80	60	1.3333

CAPACIDAD: 130

POR MENOR PESO:

Orden de los objetos: A: 40, B: 50, C: 60

El algoritmo tomaría A y toma B, da un total de 90 por lo que el objeto C de peso=60 ya no cabe dentro de la mochila ya que 90+60>130 que es la capacidad de la mochila

Al finalizar, la mochila termina con un peso de 90 y un valor total de 110

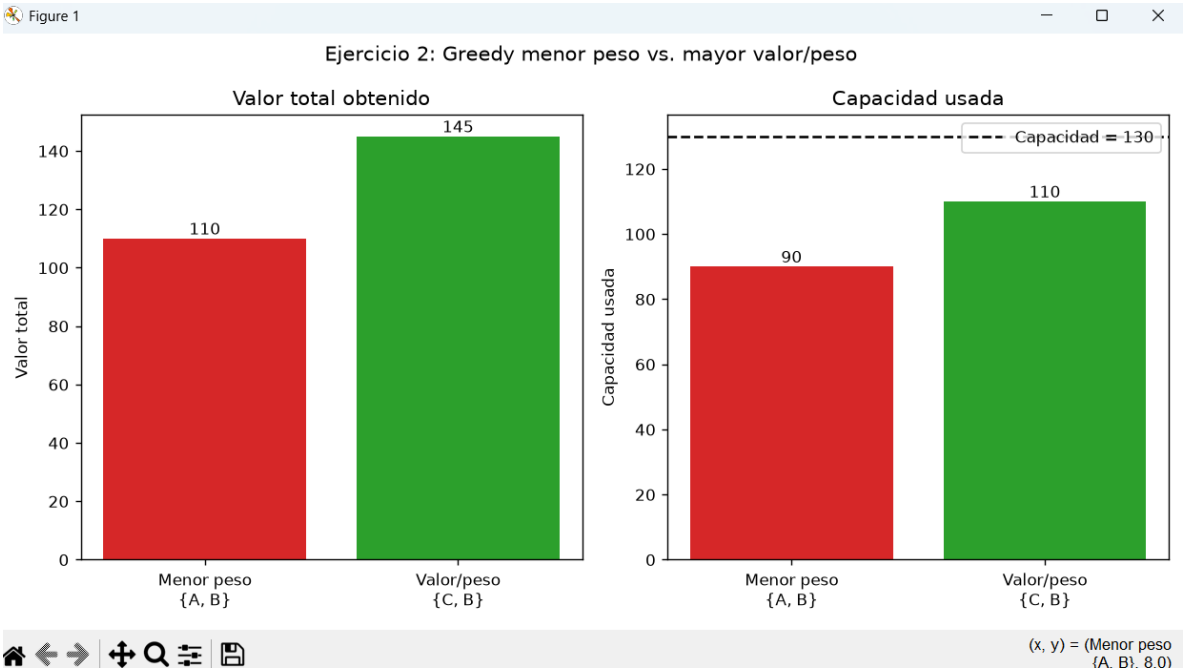
POR MAYOR RATIO VALOR/PESO:

Orden de los objetos: C:1.3333, B: 1.3, A: 1.125

El algoritmo tomaría C y toma B, da un total de 110 por lo que el objeto A ya no cabe dentro de la mochila ya que supera la capacidad de la mochila

Al finalizar, la mochila termina con un peso de 110 y un valor total de 145

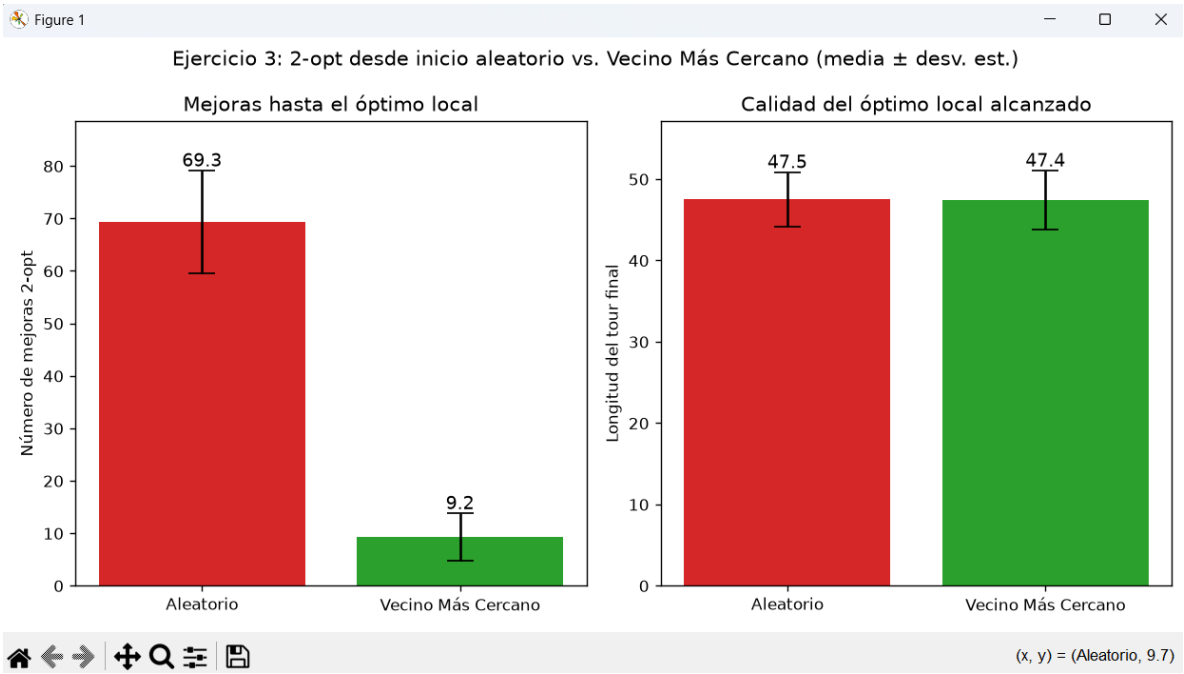
GRAFICA DE COMPARACION:



ANÁLISIS DEL EJERCICIO 3

En este ejercicio el código corre n instancias donde en cada una de estas se genera un conjunto de m ciudades, para este caso usaremos 100 instancias y 30 ciudades, en cada una de estas instancias se buscará llegar al optimo local a través de una solución inicial aleatoria, contra una solución inicial hecha con el algoritmo de Nearest Neighbor, para llegar al optimo local se usara el algoritmo de 2-opt y se contara cada mejora que se haga en cada uno de los métodos.

GRAFICA DE UNA EJECUCION CON 100 INSTANCIAS Y 30 CIUDADES:



Aquí podemos ver 2 graficas la de la izquierda contiene la comparación de mejoras que necesita cada método para llegar al optimo local, mientras que en la derecha se ve la comparación de la calidad del optimo local alcanzado.

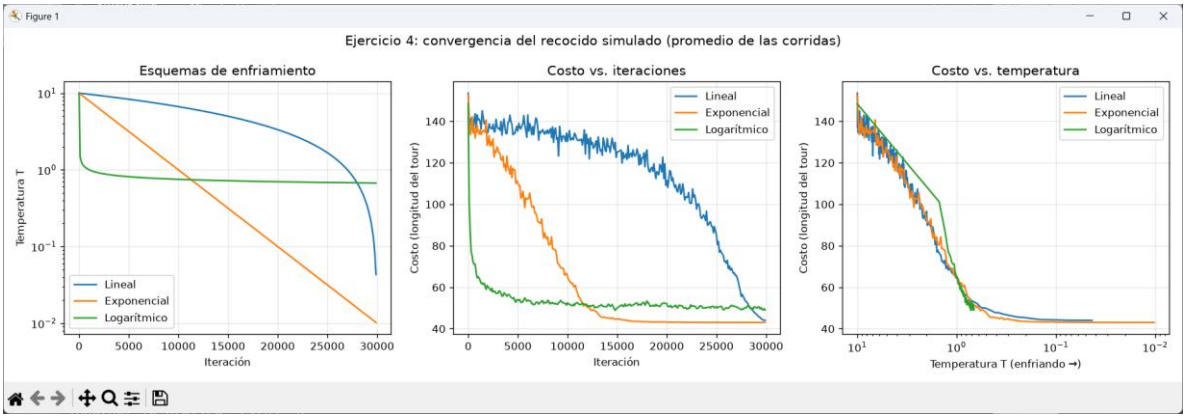
En la grafica de la izquierda es posible notar que el método de Nearest Neighbor necesita bastantes menos mejoras del 2-opt que el método por la solución inicial aleatoria, alrededor de 9 mejoras contra alrededor de 69 en promedio, esto se debe a que el método por la solución inicial aleatoria, al ser aleatorio existen muchas cruces y aristas largas que se pueden mejorar mientras que el Nearest Neighbor ordena las ciudades de forma que cada una conecte con la mas cercana, por lo que parte de un recorrido corto y con pocos cruces.

En la grafica de la derecha notamos que casi no hay cambio, esto es porque ambos métodos llegan a óptimos locales de calidad parecida, pero la solución inicial con Nearest Neighbor ahorra tiempo al necesitar menos mejoras del 2-opt.

ANALISIS EJERCICIO 4

Para este este ejercicio el código ejecuta 20 corridas sobre un conjunto de 30 ciudades utilizando diferentes presupuestos de iteraciones evaluando numéricamente el impacto de los tres esquemas de enfriamiento en cuanto a la calidad de la longitud del tour y el tiempo de ejecución en segundos.

GRAFICA DE COMPARACION DE ESQUEMAS



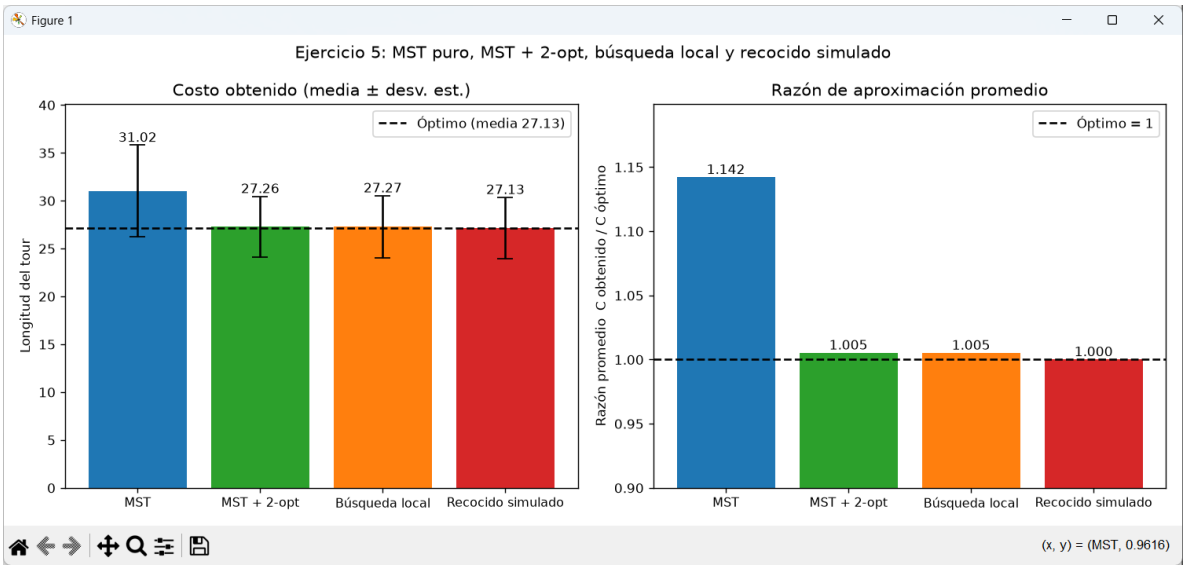
En esta grafica podemos observar el comportamiento de cada esquema a través de las diferentes cantidades de iteraciones, se observa que en el lineal la temperatura cae de forma uniforme y se aproxima a una buena solución hasta casi el final, alrededor de las 30000 iteraciones, el logarítmico baja bastante la temperatura al principio y luego empieza a bajar de forma casi imperceptible, este alcanza una solución aproximada con un presupuesto de 5000 iteraciones mientras que el exponencial baja bastante más la temperatura en las últimas iteraciones y necesita aproximadamente 15000 iteraciones para acercarse a una buena solución, a partir de ahí solo busca cambios muy pequeños.

Por lo que podemos concluir el esquema exponencial tiene el impacto más favorable y eficiente, ya que destaca por su velocidad de convergencia ofreciendo excelentes resultados incluso con presupuestos muy ajustados. No obstante, si se le otorga un presupuesto grande de 100,000 iteraciones, el tiempo de cómputo aumenta pero cualquier esquema logra exprimir el algoritmo hasta alcanzar prácticamente la misma calidad límite de solución. El tiempo de cómputo depende principalmente del número de iteraciones, ya que cada iteración hace el mismo trabajo sin importar el esquema, por lo que en general los tiempos de los tres esquemas son parecidos.

ANALISIS EJERCICIO 5

Basándonos en los resultados de la ejecución de 100 instancias de 9 ciudades evaluadas frente al óptimo por fuerza bruta, damos respuesta detallada a las preguntas planteadas

GRAFICA DE COMPARACION:



en base a la grafica se observa que la razón de aproximación se mantiene bastante alejada de 2 método MST registra una razón media de 1.142 y una razón máxima de 1.434. Esto indica que, incluso en la peor instancia de las 100 probadas, el recorrido del MST excedió en un 43.4% al óptimo real, quedando muy lejos de duplicar el costo teórico (2.0). El límite de 2 es una cota del peor caso, por lo que en instancias aleatorias el método casi nunca se acerca a él.

La aplicación de 2-opt como post-procesamiento sobre el resultado del MST mejora sustancialmente el rendimiento, reduciendo el costo medio a 27.26, situando la razón media en 1.005 e incrementando la tasa de acierto al óptimo del 5% al 80% de las instancias. Frente a la búsqueda local con inicio aleatorio , los resultados son prácticamente iguales, con una razón media de 1.005 en ambos casos y una tasa de acierto de 80% contra 77%, por lo que partir del MST no mejora el resultado final de forma apreciable. Sumado al desempeño del recocido simulado, que alcanza una efectividad perfecta del 100% en estas instancias de solo 9 ciudades, estas estrategias resultan muy precisas para este tamaño de problema, aunque con instancias más grandes los resultados podrían ser distintos.

LINK DEL REPOSITORIO:

https://github.com/Csar8008/TAREA_2__ALGORITMIA_Y_OPTIMIZACION.git

LINK DEL VIDEO:

https://youtu.be/QzAljb_YmeU